JAVA TRAINING — LESSON NOTES

Understanding *REST API*

The language microservices use to talk to each other. Before we build anything, let's understand what REST is, how HTTP works, and why it matters for the microservices architecture we learned last session.

- Prerequisites: Microservices concepts
- Level: Beginner
- Next lesson: Spring Boot

01 What is a REST API?

Let's break it down word by word:

🔌

API (Application Programming Interface)

A set of rules that allows one software application to talk to another. It defines **what you can ask for** and **what you'll get back**.

🌐

REST (Representational State Transfer)

An **architectural style** for designing APIs that use HTTP (the same protocol your browser uses). It's not a library or framework — it's a set of conventions.

So a **REST API** is a way for applications to communicate over HTTP using standard rules. When you open a website, your browser is already making REST-like calls behind the scenes.

💡 **Think of it like a restaurant waiter**

You (the **client**) don't go into the kitchen yourself. You tell the **waiter** (the API) what you want from the **menu** (the endpoints). The waiter brings your **request** to the kitchen (the **server**) and comes back with your **response** (the food). You never need to know how the kitchen works internally.

> 🔗 **Microservices Connection:** Remember how we said microservices communicate over the network? REST API is the most common way they do it. When Order Service needs to check if a product exists, it sends a REST API call to Product Service.

02 The 6 Rules of REST (Constraints)

REST isn't just "use HTTP with JSON." It's an architectural style defined by **6 constraints** (rules). If your API follows these rules, it's RESTful. If it doesn't, it's just an HTTP API. Let's understand each one with a simple analogy.

🏨 **The Hotel Analogy**

We'll use a **hotel** to explain all 6 constraints. You (the guest) are the **client**. The hotel staff is the **server**. Keep this in mind as we go through each rule.

1. Client-Server

The client and server are **separate and independent**. The client only knows how to make requests. The server only knows how to process them. They don't know each other's internal details.

Hotel: You don't enter the kitchen

As a hotel guest, you call room service and place your order. You don't walk into the kitchen to cook it yourself. The kitchen can change its entire layout, hire new chefs, or switch suppliers — **you don't care, and you don't even know**. You just call, order, and receive.

> 🔗 **Microservices Connection:** Each microservice is a server to other services. Order Service doesn't know how Product Service stores its data or what database it uses — it just calls the endpoint and gets a response. They can be built by different teams in different languages.

2. Stateless

Every request must contain **all the information** the server needs to process it. The server does not remember anything from your previous requests. No sessions, no "memory" of who you are between calls.

Hotel: Every call is a fresh call

Every time you call room service, you must say your **room number**, **your name**, and **what you want** — even if you just called 5 minutes ago. The operator doesn't remember your previous call. Why? Because there are 10 operators, and any one of them might pick up. If they all had to remember every guest's history, it wouldn't scale.

> 🔗 **Microservices Connection:** This is why microservices can scale. If Product Service has 5 running instances, any instance can handle any request because none of them need to "remember" previous calls. A load balancer can send your request to any instance — they're all interchangeable.

3. Cacheable

Responses should indicate whether they **can be cached** (saved and reused) or not. This avoids unnecessary repeated calls for the same data.

📌 **Hotel: The menu on your nightstand**

The hotel puts a **menu card** in your room. You don't call the front desk every time to ask "what food do you serve?" — you just look at the card. But if you want today's **special**, you have to call because it changes daily. The menu is *cacheable*; the daily special is *not*.

> 🔗 **Microservices Connection:** If Order Service calls Product Service to get product details, and that product rarely changes, the response can be cached. This dramatically reduces network traffic between services. Without caching, a busy system would drown in repeated identical calls.

4. Uniform Interface

All services follow the **same conventions**: standard HTTP methods, consistent URL patterns, standard status codes, and JSON format. Everyone speaks the same language.

🚪 **Hotel: Every room works the same way**

No matter which hotel room you're in — standard, deluxe, or suite — the **light switches are in the same place**, the phone works the same way, and the door lock uses the same key card. You don't need a new instruction manual for every room. The *interface is uniform*.

> 🔗 **Microservices Connection:** When every service follows the same REST conventions, any developer can understand any service's API without special training. Product Service, Order Service, Payment Service — they all use GET to read, POST to create, 404 for "not found." Predictability is power.

5. Layered System

The client doesn't know (and doesn't need to know) whether it's talking directly to the server, or to an intermediary like a load balancer, cache, or gateway. **Layers can be added without the client knowing.**

Hotel: You call one number, many people handle it

You dial **0 for the front desk**. But behind that single number, your call might be routed to the concierge, the kitchen, or the housekeeping manager. You don't know and you don't care — you just dial 0 and your request gets handled. Layers of staff work behind one simple interface.

> 🔗 **Microservices Connection:** This is exactly how the **API Gateway** works. The client calls one URL. Behind it, the gateway routes to the correct microservice, handles authentication, does load balancing — the client has no idea. You can add caching layers, security layers, or monitoring layers without changing a single line of client code.

6. Code on Demand (Optional)

The server can optionally send executable code (like JavaScript) to the client to extend its functionality. This is the **only optional constraint** and is rarely used in typical REST APIs.

Hotel: The hotel app on your phone

Some hotels ask you to download their app to control the room — lights, AC, TV. The hotel *sent code to your device* to extend what you can do. This is optional — the hotel works fine without it, but the app adds extra capabilities.

> **For your reference:** You'll rarely use Code on Demand in Java microservices. The first 5 constraints are the ones that matter day-to-day. But it's good to know it exists for completeness.

All 6 Constraints at a Glance

#	CONSTRAINT	ONE-LINE SUMMARY	HOTEL ANALOGY
1	Client-Server	Client and server are separate	You don't enter the kitchen
2	Stateless	Every request is self-contained	Say your room number every time you call
3	Cacheable	Responses can be saved and reused	Menu card in the room
4	Uniform Interface	Same conventions everywhere	Every room's light switch works the same
5	Layered System	Client doesn't see the layers behind	Dial 0, staff figures out the rest
6	Code on Demand	Server can send runnable code (optional)	Hotel app on your phone

03 Why REST for Microservices?

In a monolith, modules talk to each other through **direct method calls** — fast and simple. But in microservices, each service is a separate application running on a separate machine. They need a way to communicate **over the network**.

MONOLITH (METHOD CALLS) VS MICROSERVICES (REST API CALLS)

Why REST is the Most Popular Choice

→ **Uses standard HTTP** — the same protocol browsers use. Every language supports it.

→ **Language-independent** — Order Service (Java) can talk to Payment Service (Python) or Notification Service (Node.js). It doesn't matter.

→ **Simple and readable** — URLs describe resources, HTTP methods describe actions. Easy to understand and debug.

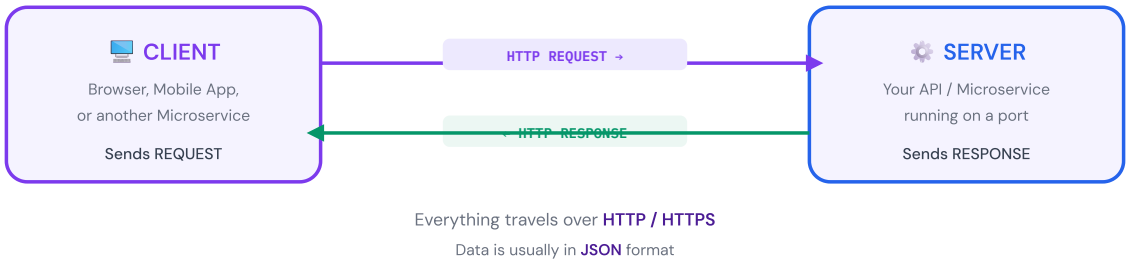
→ **Stateless** — each request contains everything the server needs. No need to "remember" previous calls.

→ **Well-supported tooling** — Postman, Swagger, curl, browser dev tools. Easy to test and document.

04 How REST API Works

Every REST API interaction follows a simple pattern: a **client** sends a **request**, and a **server** sends back a **response**.

THE REQUEST-RESPONSE CYCLE



What's Inside a Request?

PART	WHAT IT IS	EXAMPLE
HTTP Method	The action you want to perform	`GET` , `POST` , `PUT` , `DELETE`
URL (Endpoint)	The resource you want to access	`/api/products/42`
Headers	Extra info (content type, auth token)	`Content-Type: application/json`
Body	Data you're sending (for POST/PUT)	`{"name": "Laptop", "price": 999}`

What's Inside a Response?

PART	WHAT IT IS	EXAMPLE
Status Code	Did it work? What happened?	`200 OK` , `404 Not Found`
Headers	Metadata about the response	`Content-Type: application/json`
Body	The actual data returned	`{"id": 42, "name": "Laptop"}`

05 HTTP Methods (The Verbs)

HTTP methods tell the server **what action** you want to perform on a resource. Think of them as the verbs of REST.

METHOD	PURPOSE	EXAMPLE	HAS BODY?
GET	**Read** data — retrieve a resource	`GET /api/products`	No
POST	**Create** a new resource	`POST /api/products`	Yes
PUT	**Update** an existing resource	`PUT /api/products/42`	Yes
DELETE	**Delete** a resource	`DELETE /api/products/42`	No

💡 **CRUD = Create, Read, Update, Delete**

These four operations map directly to HTTP methods: **POST = Create**, **GET = Read**, **PUT = Update**, **DELETE = Delete**. Almost every API you'll ever build revolves around CRUD operations on resources.

🔗 **Microservices Connection:** When Order Service creates a new order, it might need to `GET /api/products/42` from Product Service to verify the product exists, then `POST /api/payments` to Payment Service to charge the customer. Each service exposes its own REST endpoints.

06 URL Design (Endpoints)

In REST, URLs represent **resources** (things), not actions. The URL tells you **what** you're working with, and the HTTP method tells you **what you're doing** with it.

Good URL Design Rules

→ Use **nouns**, not verbs — `/api/products` not `/api/getProducts`

→ Use **plural** names — `/api/products` not `/api/product`

→ Use **lowercase** and **hyphens** — `/api/order-items` not `/api/OrderItems`

→ Use **path parameters** for specific resources — `/api/products/42`

→ Use **query parameters** for filtering — `/api/products?category=laptops&sort=price`

→ Use **nesting** for relationships — `/api/orders/15/items` (items of order 15)

Example: Product Service Endpoints

METHOD	URL	WHAT IT DOES
GET	/api/products	Get all products
GET	/api/products/42	Get product with ID 42
GET	/api/products?category=laptops	Get products filtered by category
POST	/api/products	Create a new product
PUT	/api/products/42	Replace product 42 entirely
DELETE	/api/products/42	Delete product 42

❌ Bad vs ✅ Good URL Design

BAD (VERB-BASED)	GOOD (RESOURCE-BASED)	WHY
GET /getProducts	GET /api/products	GET already means "get" — don't repeat it

BAD (VERB-BASED)	GOOD (RESOURCE-BASED)	WHY
`POST /createUser`	`POST /api/users`	POST already means "create"
`POST /deleteProduct/42`	`DELETE /api/products/42`	Use the DELETE method, not a verb in the URL
`GET /getUserOrders/5`	`GET /api/users/5/orders`	Use nesting to show relationships

07 HTTP Status Codes

When the server responds, it includes a **status code** — a number that tells the client what happened. You don't need to memorize all of them, but you must know the common ones.

The Five Categories

RANGE	CATEGORY	MEANING
1xx	Informational	Request received, processing (rare in REST)
2xx	Success	Request worked as expected ✅
3xx	Redirection	Resource moved somewhere else
4xx	Client Error	You (the caller) made a mistake ⚠️
5xx	Server Error	The server failed internally 🔥

The Status Codes You Must Know

CODE	NAME	WHEN TO USE
200	OK	GET or PUT succeeded
201	Created	POST succeeded — new resource created
204	No Content	DELETE succeeded — nothing to return
400	Bad Request	Invalid data sent (missing fields, wrong format)
401	Unauthorized	No authentication — who are you?
403	Forbidden	Authenticated but not allowed to do this
404	Not Found	The resource doesn't exist
409	Conflict	Request conflicts with current state (e.g., duplicate)
500	Internal Server Error	Something went wrong on the server (bug, crash)
503	Service Unavailable	Server is overloaded or under maintenance

> 🔗 **Microservices Connection:** Status codes are critical for microservices. If Order Service calls Product Service and gets a `404`, it knows the product doesn't exist. If it gets a `503`, it knows Product Service is down and can trigger a **fallback** or **retry** (remember circuit breakers from the microservices lesson?).

08 Request & Response in Detail

Let's look at a complete example. We'll use the **Product Service** from our microservices example.

Example 1: GET a Product

REQUEST

```
GET /api/products/42 HTTP/1.1
Host: product-service:8082
Accept: application/json
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 42,
  "name": "Wireless Keyboard",
  "price": 49.99,
  "category": "electronics",
  "inStock": true
}
```

Example 2: POST (Create) a New Product

REQUEST

```
POST /api/products HTTP/1.1
Host: product-service:8082
Content-Type: application/json

{
  "name": "USB-C Hub",
  "price": 29.99,
  "category": "accessories"
}
```

RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
Location: /api/products/43

{
  "id": 43,
  "name": "USB-C Hub",
  "price": 29.99,
  "category": "accessories",
  "inStock": true
}
```

Example 3: DELETE a Product

REQUEST

```
DELETE /api/products/42 HTTP/1.1
Host: product-service:8082
```

RESPONSE

```
HTTP/1.1 204 No Content

(empty body — the product has been deleted)
```

Example 4: Error Response

REQUEST

```
GET /api/products/999 HTTP/1.1
Host: product-service:8082
```

RESPONSE

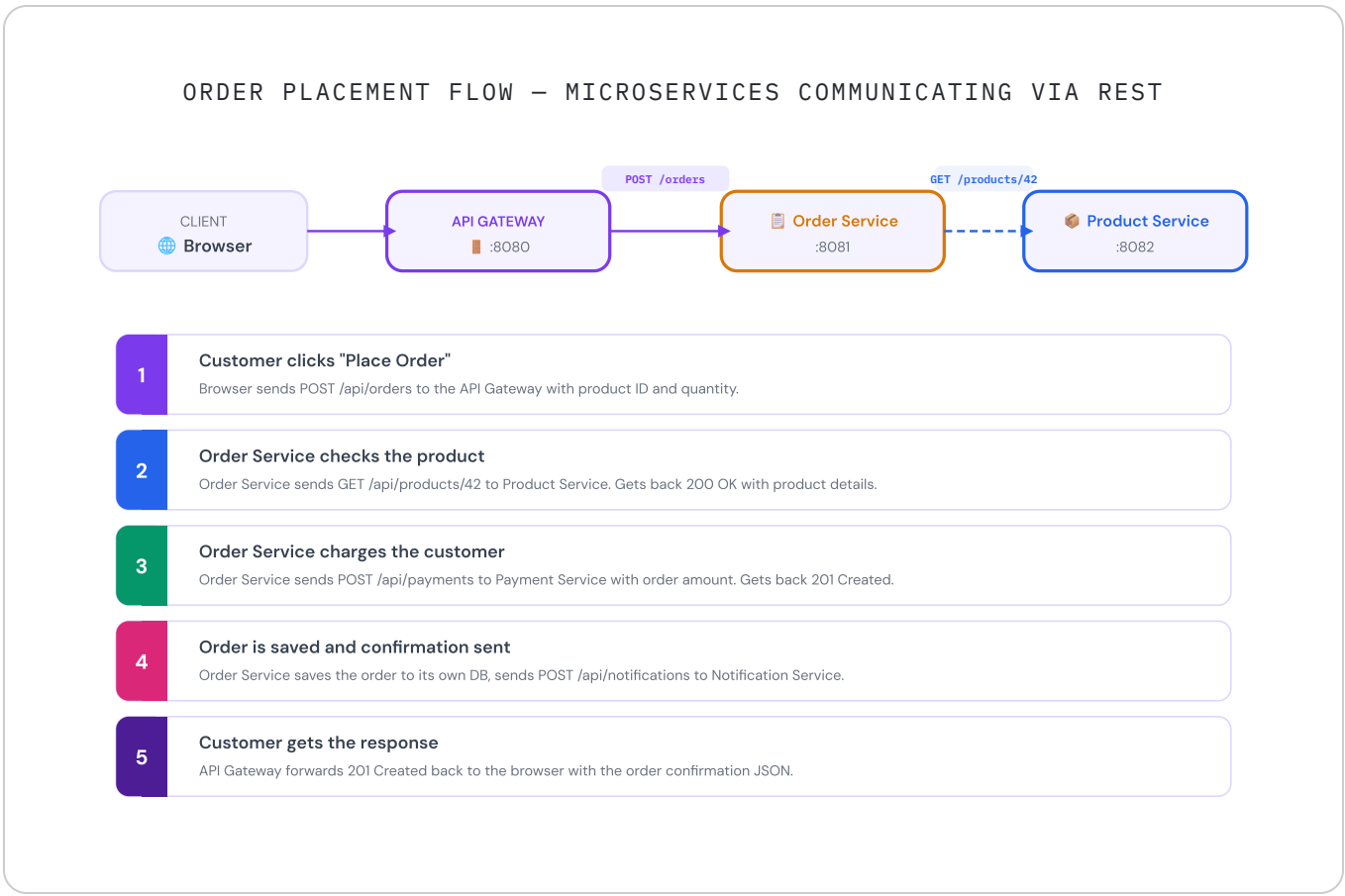
```
HTTP/1.1 404 Not Found
Content-Type: application/json

{
```

```
  "error": "NOT_FOUND",
  "message": "Product with id 999 does not exist",
  "timestamp": "2026-05-29T10:30:00Z"
}
```

09 How Microservices Talk via REST

This is where it all comes together. Let's walk through a **real scenario**: a customer places an order on your e-commerce site.



> **Notice:** Every step above is a **REST API call** between independent services. Each service has its own endpoints, its own database, and only knows what it needs to. This is microservices + REST in action.

Each Service Exposes Its Own REST Endpoints

SERVICE	BASE URL	EXAMPLE ENDPOINTS
Product Service	`:8082/api/products`	GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}
Order Service	`:8081/api/orders`	GET /, GET /{id}, POST /, GET /user/{userId}
Payment Service	`:8083/api/payments`	POST /, GET /{id}, GET /order/{orderId}
User Service	`:8084/api/users`	GET /{id}, POST /, PUT /{id}, POST /login
Notification Service	`:8085/api/notifications`	POST /, GET /user/{userId}

10 JSON — The Data Format

REST APIs almost always use **JSON** (JavaScript Object Notation) to send and receive data. It's lightweight, human-readable, and supported by every programming language.

JSON Basics

JSON

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "price": 49.99,
  "inStock": true,
  "tags": ["electronics", "peripherals"],
  "manufacturer": {
    "name": "TechCo",
    "country": "Japan"
```

```
  }
}
```

JSON TYPE	EXAMPLE	JAVA EQUIVALENT
String	`"hello"`	`String`
Number	`42` , `3.14`	`int` , `double`
Boolean	`true` , `false`	`boolean`
Null	`null`	`null`
Array	`["a", "b"]`	`List<String>`
Object	`{"key": "value"}`	`Map` or custom class

> **Why JSON over XML?** JSON is shorter, easier to read, and faster to parse. While older APIs used XML, virtually all modern REST APIs use JSON. In Java, libraries like Jackson and Gson handle JSON-to-Object conversion automatically.

11 REST API Best Practices

Version Your API

Use `/api/v1/products` so you can release v2 without breaking existing clients. Critical when multiple microservices depend on your endpoints.

Use Consistent Error Responses

Always return errors in the same JSON structure: `{"error": "...", "message": "...", "status": 404}` .

This makes error handling predictable across all services.

Secure Your Endpoints

Use HTTPS always. Authenticate with tokens (JWT). Not every service should be publicly accessible — internal services can be behind the API Gateway.

Document Your API

Use OpenAPI/Swagger to generate documentation. In microservices, teams depend on each other's APIs — good docs prevent miscommunication.

Use Pagination

Never return all records at once. Use `?page=1&size=20`. Imagine Product Service has 100,000 products — sending all of them in one response would be a disaster.

Make It Stateless

Each request must contain everything the server needs. Don't rely on "sessions" between calls. This is what makes microservices scalable — any instance can handle any request.

12 Summary

What to take away from this lesson

1. REST API is how applications talk to each other over HTTP — it's the main communication method for microservices.
2. HTTP methods define the action: GET (read), POST (create), PUT (update), DELETE (remove).
3. URLs represent resources (nouns), not actions (verbs). Use plural, lowercase, hyphenated.

4. Status codes tell the client what happened: 2xx = success, 4xx = your fault, 5xx = server's fault.

5. JSON is the standard data format for REST APIs.

6. In microservices, each service exposes its own REST endpoints and calls other services' endpoints over the network.

7. *Next lesson:* We'll build actual REST APIs using **Spring Boot** — turning these concepts into working Java code.
